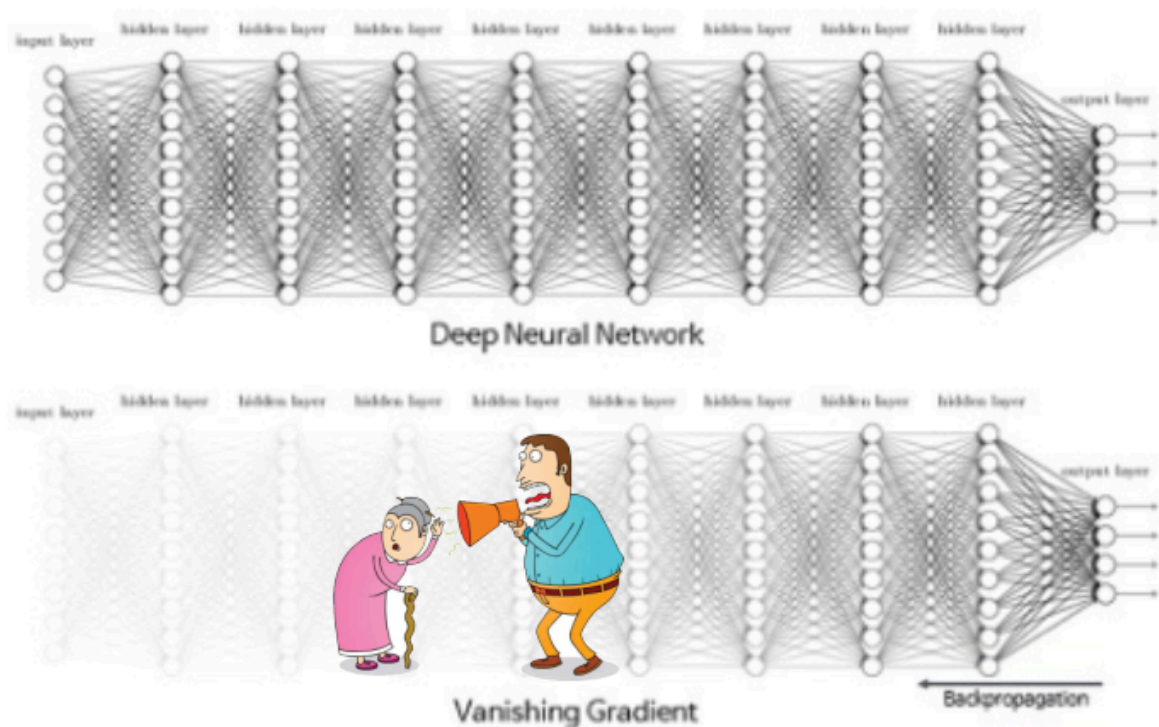


# Desvanecimiento y explosión del gradiente

## Desvanecimiento del gradiente

El **problema del vanishing gradient** (gradiente que se desvanece) ocurre durante el entrenamiento de redes neuronales profundas, especialmente cuando usamos métodos como **backpropagation** para ajustar los pesos.

Sucede cuando los gradientes (derivadas parciales de la función de pérdida respecto a los pesos) se vuelven extremadamente pequeños al propagarse desde las capas de salida hacia las capas de entrada. Esto hace que las capas más cercanas a la entrada se actualicen muy lentamente o no se actualicen en absoluto, lo que ralentiza o detiene el aprendizaje.



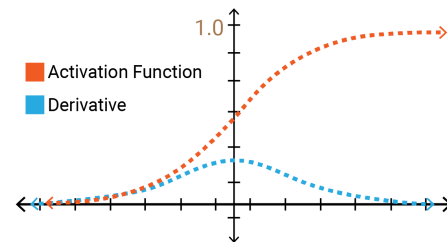
### ¿Por qué ocurre?

1. Funciones de activación saturantes:

- Funciones como **sigmoide** o **tanh** comprimen los valores de entrada en un rango limitado (ej: 0-1 para sigmoide). Cuando las entradas son muy grandes o muy pequeñas, sus derivadas tienden a cero. Al multiplicar estas derivadas pequeñas durante backpropagation, los gradientes se reducen exponencialmente en capas anteriores.

Ejemplo:

Si usamos sigmoide, su derivada máxima es 0.25. En una red con 10 capas, el gradiente podría multiplicarse por  $0.25^{10}$ , resultando en un valor casi cero.



## 2. Redes muy profundas:

- Cuanto más profunda es la red, más capas hay por las que se propagan los gradientes. Esto multiplica el riesgo de que los valores se reduzcan drásticamente.

## 3. Inicialización inadecuada de pesos:

- Pesos inicializados con valores muy pequeños o muy grandes pueden exacerbar el problema. Por ejemplo, pesos pequeños amplifican la saturación de las funciones de activación.

---

## Consecuencias

- Las capas iniciales (cercanas a la entrada) aprenden muy lentamente o se estancan, mientras que las capas finales (cercanas a la salida) se actualizan normalmente.
  - La red no puede capturar patrones complejos o relaciones a largo plazo en los datos.
  - En redes recurrentes (RNN), este problema dificulta aprender dependencias temporales largas (ej: en lenguaje natural o series temporales).
- 

## Soluciones comunes

## 1. Funciones de activación no saturantes:

- **ReLU (Rectified Linear Unit):**  $f(x) = \max(0, x)$ . Su derivada es 1 para  $x > 0$ , lo que evita el desvanecimiento en capas positivas. Variantes como Leaky ReLU o ELU también ayudan.
- **Softplus:** Una alternativa suave a ReLU que evita el "neurona muerta" (problema en ReLU cuando no se activa nunca).

## 2. Inicialización adecuada de pesos:

- **Inicialización de Xavier/Glorot:** Escala los pesos según el número de neuronas en cada capa, manteniendo varianza estable.
- **Inicialización de He:** Similar a Xavier, pero diseñada específicamente para ReLU.

## 3. Arquitecturas con conexiones residuales (ResNet):

- Usan **skip connections** (conexiones directas) que permiten que el gradiente fluya sin pasar por todas las capas, mitigando el desvanecimiento.

## 4. Batch Normalization:

- Normaliza las entradas de cada capa, reduciendo la *covariate shift* y ayudando a mantener gradientes estables.

## 5. Redes recurrentes especializadas:

- **LSTM (Long Short-Term Memory) o GRU (Gated Recurrent Units):** Incluyen puertas (gates) que controlan el flujo de información, permitiendo gradientes que persisten durante más pasos temporales.

## 6. Regularización y ajuste de hiperparámetros:

- Reducir la profundidad de la red, usar tasas de aprendizaje adecuadas o técnicas como **gradient clipping** (para evitar explosión de gradientes, un problema opuesto).

---

El **problema de la explosión de gradientes** (gradient explosion) es el fenómeno opuesto al vanishing gradient. Ocurre cuando los gradientes (derivadas parciales de la función de pérdida respecto a los pesos) crecen exponencialmente durante la retropropagación (**backpropagation**), lo que lleva a actualizaciones de pesos extremadamente grandes. Esto hace que el

entrenamiento de la red neuronal sea inestable, con valores que pueden llegar a infinito (NaN) o divergir completamente.

---

# Explosión del Gradiente

## ¿Por qué ocurre?

### 1. Inicialización inadecuada de pesos:

- Si los pesos iniciales son demasiado grandes, las salidas de las neuronas también lo serán. Al propagar estos valores hacia adelante y hacia atrás, los gradientes se amplifican en cada capa.

### 2. Redes muy profundas:

- En redes con muchas capas, los gradientes se multiplican a través de todas ellas. Si cada gradiente es mayor que 1, su producto final puede volverse gigantesco (ej:  $2^{10} = 1024$  para 10 capas).

### 3. Funciones de activación no saturantes:

- Funciones como **ReLU** ( $f(x) = \max(0, x)$ ) tienen una derivada constante de 1 para  $x > 0$ . Esto evita el desvanecimiento, pero si los pesos son grandes, las salidas de las capas anteriores pueden crecer sin control, causando gradientes explosivos.

### 4. Redes recurrentes (RNN):

- En RNN, los gradientes se propagan no solo a través de capas, sino también a través de pasos temporales. Si las dependencias son largas, los gradientes pueden acumularse y explotar (ej: en lenguaje natural o series temporales).

### 5. Tasa de aprendizaje alta:

- Una tasa de aprendizaje excesiva amplifica aún más los gradientes durante las actualizaciones de pesos, exacerbando la inestabilidad.

---

## Consecuencias

- **Pesos que divergen:** Los valores de los pesos se vuelven NaN o infinitos, rompiendo el entrenamiento.

- **Pérdida (loss) inestable:** La función de pérdida oscila violentamente o crece sin control.
  - **Imposibilidad de convergencia:** El modelo no puede aprender patrones útiles debido a actualizaciones caóticas.
- 

## Soluciones comunes

### 1. Clipping de gradientes (Gradient Clipping):

- **Límite por norma:** Se establece un umbral máximo para la norma del gradiente. Si el gradiente excede este valor, se escala proporcionalmente.

Ejemplo:

`clipnorm=1.0` limita la norma del gradiente a 1.0.

- **Límite por valor:** Se recortan los valores individuales del gradiente que superen un umbral.

Ejemplo:

`clipvalue=0.5` asegura que ningún gradiente supere  $\pm 0.5$ .

### 2. Inicialización adecuada de pesos:

- Usar inicializaciones como **Xavier** (para sigmoide/tanh) o **He** (para ReLU) para mantener las varianzas de las activaciones y gradientes estables.

### 3. Normalización de capas:

- **Batch Normalization** o **Layer Normalization** estandarizan las entradas de cada capa, reduciendo la covariate shift y estabilizando los gradientes.

### 4. Arquitecturas especializadas:

- **LSTM/GRU** en RNN: Sus puertas (gates) controlan el flujo de información, mitigando la acumulación de gradientes en pasos temporales.
- **ResNets:** Las conexiones residuales permiten que los gradientes fluyan directamente sin pasar por todas las capas.

### 5. Regularización:

- **Regularización L2 (weight decay):** Penaliza pesos grandes, evitando que se amplifiquen los gradientes.

- **Dropout:** Reduce la dependencia de neuronas individuales, promoviendo robustez.

## 6. Ajuste de la tasa de aprendizaje:

- Usar tasas de aprendizaje más bajas o técnicas adaptativas como **Adam**, que ajustan dinámicamente la tasa según la magnitud de los gradientes.

---

## Ejemplo práctico

Imagina una RNN entrenando un modelo de lenguaje:

- Sin clipping, los gradientes acumulados a través de 50 pasos temporales podrían explotar, causando valores NaN.
- Aplicando `clipnorm=5.0`, los gradientes se escalan si su norma supera 5, permitiendo actualizaciones controladas y estables.

Otro ejemplo: una red profunda con 20 capas usando ReLU. Si los pesos iniciales son muy grandes:

- Las activaciones y gradientes crecen exponencialmente, llevando a divergencia.
- Cambiando a inicialización He y añadiendo batch normalization, los gradientes se estabilizan y el entrenamiento converge.

---

## Contraste con el vanishing gradient

- **Vanishing Gradient:**

Gradientes  $\rightarrow 0 \rightarrow$  capas iniciales no aprenden.

Solución: ReLU, inicialización adecuada, ResNets.

- **Exploding Gradient:**

Gradientes  $\rightarrow \infty \rightarrow$  entrenamiento inestable.

Solución: Clipping, batch normalization, LSTM/GRU.

Ambos problemas son desafíos clave en redes profundas, pero requieren estrategias opuestas para mitigarlos. Mientras el vanishing gradient limita la capacidad de aprendizaje, el exploding gradient amenaza la estabilidad del modelo.

---